

第3章. 混合检索与查询优化：BM25、RRF、Reranker

怎样找全资料，再把相关资料排到前面？

学习目标

1. 比较 Dense 与 BM25，并手算 RRF 排名融合。
2. 理解召回、精排与分数阈值的不同职责。
3. 追踪意图抽取、查询改写与元数据过滤。
4. 分析过滤零命中回退及可验证的调优方法。

阅读方法：先看业务问题和图，再读名词与源码，最后复制完整实验验证。代码节选用于定位实现；完整实验会明确运行条件。

代码基准：2026-10-03 本地项目当前源码。默认配置与环境覆盖值可能不同；本文不以旧实施计划代替真实实现。

运行约定：命令从项目根目录执行。学习实验代码直接收录在 PDF 内；自行新建练习文件后复制运行，不另附脚本。服务实验的结果应由你实际运行记录。

学习产出：每章至少留下一份自己的输入、输出和解释。涉及检索质量、耗时或简历中的改善幅度，只采用有证据的数据。

本章目录

3.1 本章要解决什么问题 3

3.2 Dense 与 BM25：为什么要两路检索 3

3.3 RRF：把两张排名表合在一起 4

3.4 Milvus 混合检索：两路请求怎样组合 6

3.5 Reranker：召回以后重新判断相关性 8

3.6 retrieve_docs：把检索组件串起来 10

3.7 Structured Output：把用户问题变成 SearchIntent 10

3.8 Query Rewrite：改写表达，保留具体编码 12

3.9 Metadata Filter：把明确条件变成检索范围 13

3.10 过滤回退：精排后零条，才做最终判断 15

3.11 实验、调试与输出解释 17

3.12 常见问题与易错点 17

3.13 本章练习与源码阅读索引 18

本章围绕车辆电路图资料目录检索展开。先理解“怎么找”，再理解“怎么排”，最后学习如何把口语问题变成可检索、可过滤的查询。项目源码是实现依据，课程材料用于组织讲解。

3.1 本章要解决什么问题

3.1.1 从一个真实需求开始

用户输入：“帮我找东风天龙 EDC17C53 电脑板针脚图。”

这句话包含品牌、车型、控制器编码和资料类型，也包含“帮我找”这样的口语。资料标题则可能写成“东风天龙 EDC17C53 发动机 ECU 针脚定义”。如果只做逐字匹配，“电脑板”和“ECU”的表达差异可能使结果漏掉；如果只做语义检索，EDC17C53 与另一个相近控制器编码又可能被混在一起。

项目希望同时利用两种能力：语义相似帮助理解表达差异，关键词匹配帮助保留品牌和编码线索。把两路结果合并以后，再用一个更精细的模型判断“这个查询和这条资料是否相符”。这就是本章的主线。

当前检索对象是从 CSV 整理出来的资料记录。每条记录的检索文本主要来自标题与层级路径。本章所说的相关性是“资料记录与查询的相关性”；它不代表已读取电路图全部内容，更不代表已验证某个针脚的技术结论。

3.1.2 学习目标

学完后，你能独立解释 Dense、BM25、RRF 和 Reranker 的分工；沿源码追踪三个不同的 Top K；说明结构化意图、查询改写和硬过滤如何配合；定位“有召回但精排后零条”为什么仍会触发过滤回退。

阅读时坚持一个习惯：每看到一个分数，先问它属于哪一阶段；每看到一个数量，先问它限制哪一层。检索链路中，同名的 score 与 top_k 经常承担不同含义。

3.2 Dense 与 BM25：为什么要两路检索

3.2.1 Dense Retrieval：稠密向量检索

Dense Retrieval 将文本编码为固定维度的稠密向量，通过向量相似度寻找候选。项目使用 BGE-M3 生成向量，Dense 请求采用 COSINE 指标。查询进入搜索前，执行 `embeddings.embed_query(query)`；资料向量则在建立索引时生成。

这里的“稠密”指向量中通常有很多非零元素。向量的某一个维度并不直接等于某个汉字。模型把文本表达压缩进整个向量，因此不同用词的相关表达有机会靠近。

例如“显示器的线怎么接”和“仪表显示器针脚定义”可能具有相关语义。Dense 可以帮助召回候选，但不能替用户确认两者就是同一块显示器。品牌、型号和文档类型仍需结合其他线索判断。

3.2.2 Sparse Retrieval：稀疏检索与 BM25

Sparse Retrieval 使用只有少量非零元素的表示。项目的稀疏路由 Milvus 内置 BM25 函数完成，不是把 BGE-M3 的全部可用输出都接入了项目。

BM25 根据词项匹配、词在资料中的出现情况、词在整个集合中的稀有程度以及文本长度等因素排序。直观理解：越有区分度的词，对相关资料的加分越有价值；同一个词重复很多次，通常不会无限线性加分。

对于 EDC17C53、XE135GII 这样的编码，保留原始文本很有价值。但“BM25 擅长关键词”不等于“保证编码精确匹配”。是否切成完整编码，还受中文分析器和分词规则影响；最终应检查真实查询结果。代码配置的是 `analyzer_params={"type": "chinese"}`。

3.2.3 两路检索的职责对照

比较项	Dense 路线	BM25 路线
主要依据	文本向量的语义相似度	词项匹配与统计权重
项目输入	查询文本生成的向量	原始查询文本
Milvus 字段	dense	sparse
当前指标	COSINE	BM25
常见价值	补充不同表达的相关候选	保留品牌、编码、资料名称线索
主要限制	相近语义可能混入不同型号	表达差异或分词可能导致漏召回

应用场景。 当资料库同时存在口语表达、专业术语和字母数字编码时，混合检索值得学习。如果资料本身字段很规范、需求又是明确编号查询，应优先判断是否直接按 ID 或标量字段查询，不能因为学了向量检索就把所有查询都变成语义搜索。

3.3 RRF：把两张排名表合在一起

3.3.1 原始分数为什么不能直接相加

Dense 返回的相似度与 BM25 的词项统计分数不是同一种量。假设 Dense 给资料 A 打 0.91 分，BM25 给资料 B 打 12 分，不能由此认定 B 的相关性是 A 的十几倍。尺度不同，直接相加会让数值范围更大的一路支配结果。

RRF 的全称是 Reciprocal Rank Fusion，中文可理解为倒数排名融合。它先忽略两路分数的数值尺度，只使用“资料在各路结果中排第几”。某条资料在两路都靠前，累加后通常得到更高融合分数。

3.3.2 公式中的 k 到底是什么

对于资料 d，RRF 分数等于它在每一路结果中的贡献之和。某一路排名为 r 时，贡献是 $1 / (k + r)$ ；排名从 1 开始，没出现在某一路就不贡献那一路分数。

项目默认 `rrf_k=60`。这个 k 是平滑常数，不是返回 60 条资料。它调节排名靠前与靠后的差距。设 k 为 60，第 1 名贡献约 0.016393，第 2 名贡献约 0.016129；两者都很小，并且不能解释为相关概率。

要特别区分三个名称：RRF 的 k 调节排名贡献，候选数量 `candidate_k` 控制每路召回规模，最终 `top_k` 控制某一阶段返回上限。三者写在一起，职责仍然不同。

3.3.3 完整离线实验：观察排名如何影响融合

下面是可完整运行的教学实验。使用项目纯函数，不初始化向量模型，不连接 Milvus。请在项目根目录打开 Python 解释器，复制整段代码执行。

代码块 python

```
1 from app.rag.fusion import reciprocal_rank_fusion
2 from app.schemas.search import SearchResult
3
4 def doc(doc_id, score):
5     return SearchResult(
6         doc_id=doc_id, title=f"D{doc_id}",
7         hierarchy_path="p", content=f"D{doc_id}",
8         score=score,
9     )
10
11 dense = [doc(1, .91), doc(2, .86), doc(3, .77)]
12 sparse = [doc(2, 12), doc(4, 7), doc(1, 3)]
13 results = reciprocal_rank_fusion([dense, sparse], k=60)
14 for item in results:
15     print(item.doc_id, round(item.score, 6))
```

本次实际输出：

代码块 text

```
1 2 0.032522
2 1 0.032266
3 4 0.016129
4 3 0.015873
```

资料 2 在 Dense 排第 2，在 BM25 排第 1，因此贡献是 $1/62 + 1/61$ 。资料 1 在 Dense 排第 1，在 BM25 排第 3，因此贡献是 $1/61 + 1/63$ 。资料 2 稍高，虽然它的 Dense 原始分数比资料 1 低。

把 sparse 中的分数 12 改成 1200，保持列表次序不变，RRF 结果也不变。这是理解“基于排名融合”的关键实验。实验数据是人为构造的四条记录，输出只能验证融合算法，不是项目真实检索质量。

3.3.4 独立融合函数与 Milvus 内置融合

项目同时保留两条与 RRF 有关的实现。app/rag/fusion.py 的 reciprocal_rank_fusion() 接收 Python 结果列表，适合离线实验、独立列表融合和单元验证。app/rag/hybrid_search.py 则把 RRFRanker 对象交给 Milvus 的 hybrid_search()，由数据库服务对多路请求进行融合。

主检索链路采用后者。不能看到仓库有一个 Python RRF 函数，就在流程图中写成“Milvus 融合后再调用 fusion.py 融合一次”。目前 retrieve_docs() 没有这样调用。

独立函数还按 doc_id 聚合，后出现的同 ID 对象会成为最终对象内容来源。它假设每一路已按相关性排序，而且一路内不应反复出现同 ID。学习时可以讨论这些输入约束，生产接入时应对齐结果来源与去重规则。

3.4 Milvus 混合检索：两路请求怎样组合

3.4.1 从文档索引到检索请求

上一章建立的混合 Collection 同时包含 dense 与 sparse 两个向量字段。查询时，项目不会请求两个不同的聊天模型搜索资料，而是创建两份 AnnSearchRequest：Dense 请求携带查询向量，Sparse 请求携带查询文本。

两份请求使用相同的业务过滤条件，分别召回候选；Milvus 按 RRF 融合；融合结果再交给 Reranker。这是“多路召回”，不是“两个 Agent 协作”。



图3-1 两路召回、RRF 融合与精排。

Dense 与 BM25 使用不同证据；精排处理召回候选。图中数量为 retrieve_docs 默认路径。

3.4.2 源码节选：创建 Dense 与 Sparse 请求

下列为 app/rag/hybrid_search.py 中的源码节选，依赖该函数上文的变量，不应整段作为独立脚本执行。

代码块 python

```
1 dense_request = AnnSearchRequest(
2     data=[query_dense_vector],
3     anns_field="dense",
4     param={"metric_type": "COSINE"},
5     limit=candidate_k,
6     filter=filter_query,
7 )
8
9 sparse_request = AnnSearchRequest(
10    data=[query],
11    anns_field="sparse",
12    param={"metric_type": "BM25"},
13    limit=candidate_k,
14    filter=filter_query,
15 )
16
17 reqs = [dense_request, sparse_request]
```

注意两处 data 不相同。Dense 是向量，Sparse 是文本；它们都用列表包装，因为 Milvus 接口支持批量查询。项目目前按单 query 的第一组 hits 解析响应。

同样，filter 放在两路请求上，表示候选范围先受到业务条件限制。它不是最后展示结果时才执行的字符串检查。某条资料如果被过滤挡在外面，后面的 Reranker 就没有机会重新把它找回来。

3.4.3 默认数量：不同入口走不同路径

下面是 app/core/config.py 的默认值。env 或运行时参数可以覆盖它们，因此阅读源码默认值与判断本机实际设置要分开。

设置项	默认值	实际限制的位置
dense_top_k / sparse_top_k	20 / 20	单独 Dense、BM25 搜索入口
hybrid_candidate_k	20	混合检索每一路的候选上限
hybrid_top_k	10	直接 hybrid_search 的融合返回上限
reranker_top_k / reranker_min_score	5 / 0.0	通用精排返回上限与严格分数阈值
graph_candidate_k / graph_max_results	30 / 5	Web 工作流保留候选与最终展示分支
rrf_k / reranker_batch_size	60 / 16	RRF 平滑常数与每批精排规模

直接调用 hybrid_search(query)，默认每路取 20、融合后最多返回 10。调用 retrieve_docs(query) 时，它把候选上限 20 同时传给混合检索的 candidate_k 和 top_k，先保留最多 20 个融合候选，再精排取最多 5。

Web 工作流的 retrieve 节点则每路最多 30，融合最多 30，精排后暂时也最多保留 30，以供后续分类澄清。因此“项目只检索 5 条”是不准确的。5 是默认最终展示分支的限制，不是整个召回过程的候选规模。

3.4.4 响应数据怎样变成统一结果

Milvus 返回实体字段与排名分数。_parse_results() 把它们整理成 HybridSearchResult，保留 doc_id、title、hierarchy_path、content 和 score。

这层转换让后续算法不用一直理解数据库原始响应。比如调用精排时，只需要读取 doc.content，最终展示时读取 doc.title 和 doc.doc_id。统一结果结构是模块之间的接口，不是为了多写几个类。

3.5 Reranker：召回以后重新判断相关性

3.5.1 为什么召回后还要排序

召回阶段倾向于快速找到一批可能相关的资料。两个向量独立编码后的相似度比较很快，但对查询与资料的细节交互判断有限。精排阶段把查询和候选内容成对输入模型，让模型共同检查这两段文本的匹配关系。

Reranker 中文通常叫重排模型或精排模型；Cross-Encoder 是交叉编码器。相比可预先计算资料向量的 Bi-Encoder，Cross-Encoder 通常需要对每个 query-document pair 重新计算。因此它放在候选召回之后，避免对整个资料集合逐条做模型推理。

如果正确资料根本没有被召回，Reranker 无法凭空补出它。优化精排前要先检查召回率；否则只是把错误候选排得更整齐。

3.5.2 当前项目怎样调用 Qwen3 Reranker

模型名默认是 Qwen/Qwen3-Reranker-0.6B。get_reranker_model() 使用 Sentence Transformers 的 CrossEncoder 初始化模型，通过缓存复用实例。设备配置为 auto 时，检测到 CUDA 可用就选择 cuda，否则选择 cpu。

当前业务源码的核心调用如下。它是源码节选，docs 与 settings 来自该函数上下文。

代码块 python

```
1 model = model or get_reranker_model()
2 pairs = [(query, doc.content) for doc in docs]
3 scores = model.predict(
4     pairs,
5     batch_size=settings.reranker_batch_size,
6 )
```

这里一个重要细节是：项目并没有自己编写 Qwen3 的全部 token 推理代码，而是通过 CrossEncoder 接口调用。阅读项目时，先理解它给库传入了什么，再追踪库如何解释这组输入。

本次本机安装的 Sentence Transformers 版本为 5.7.0。其 CrossEncoder 加载逻辑对 ForCausalLM 架构会选用文本生成模型，并配置 yes、no token 的 LogitScore 模块。这说明当前安装库存在生成式模型的重排适配路径，不能笼统认为 CrossEncoder 只支持序列分类模型。依赖版本升级或换环境后，需要重新核对此一层。

3.5.3 Qwen3 的任务提示与 token scoring

Qwen3 Reranker 官方模型卡提供的评分思路是：给模型明确的相关性判断任务，把 Instruction、Query 和 Document 组合为任务输入，使用符合模型要求的消息模板，让模型判断候选能否满足查询，再取最后位置对应 yes 与 no 的 logits。

Logit 是模型在归一化之前给某个 token 的原始数值，不是已经计算好的概率。把 yes 与 no 两个数做 softmax，yes 的归一化概率可以作为相关性分数。两个 token 的 logit 差经过 sigmoid，也得到同样的二项归一化结果。

完整的输入模板、特殊 token、截断规则和实际分数由模型、库版本及配置共同决定。项目业务代码没有显式传入 Qwen 专用 prompt；加载器可能使用模型保存的模板。只读 predict() 一行，不能进一步宣称已与官方模板完全一致。这里没有下载权重或执行真实 Qwen 推理，实际模板与效果仍需在读者的本地运行中核对。

完整离线数学实验。 此例只说明两个 logits 怎样归一化，不调用任何模型。

代码块 python

```
1 from math import exp
2
3 yes_logit = 2.0
4 no_logit = 0.5
5 delta = yes_logit - no_logit
6 probability = 1.0 / (1.0 + exp(-delta))
7 print(round(delta, 6))
8 print(round(probability, 6))
```

数学结果为 1.5 和 0.817574。后者表示模型在这两个选项之间的归一化倾向，不能当作该电路资料有 81.76% 的事实正确率。任务模板变化会影响判断，分数尺度变化也会影响阈值的意义。

3.5.4 精排后的排序与阈值

业务代码把预测分数写入 RerankResult.score，把原来的融合分数另存到 retrieval_score。随后按新分数降序排列，只保留 score > min_score 的记录，最后切片取 top_k。

注意比较是严格大于，不是大于等于。默认 min_score 为 0.0，恰好为 0 的记录被丢弃。如果模型输出是接近零但大于零的概率，很多记录可能仍保留；如果输出是 logit 差，同样的阈值就具有另一层含义。因此默认 0.0 只是代码配置，不是已通过业务评估选出的最优界线。

结果字段	产生阶段	适合用于什么判断
DenseSearchResult.score	Dense 检索	同一次 Dense 返回中的相似度排序
SparseSearchResult.score	BM25 检索	同一次 BM25 返回中的词项相关排序
HybridSearchResult.score	RRF 融合	合并候选的排名贡献
RerankResult.retrieval_score	精排前保留	回看该候选的融合分数
RerankResult.score	精排模型	最终重排与当前阈值过滤

这张表在调试中很实用。某个结果精排分数高而融合分数低，并不一定是异常，它可能正是重排模型修正召回排序的结果。应观察资料内容是否匹配，再决定分数是否符合业务需求。

3.6 retrieve_docs：把检索组件串起来

3.6.1 一个检索入口的完整职责

app/rag/pipeline.py 中的 retrieve_docs() 负责 Dense、BM25、RRF、Cross-Encoder 的串联。它会清理空白查询，读取默认数量，创建 RRFRanker，调用混合检索，再调用精排并返回。

它不会自动解析品牌，也不会自行调用查询改写。调用者传入什么 query，它就用什么 query；传入什么 filter_query，它就把过滤条件带给混合检索。这种边界设计便于其他模块复用同一套检索能力。

如果把意图解析、改写、检索、精排都统称为“RAG 函数”，学习时容易错过接口责任。更准确的阅读顺序是：查询准备决定怎么搜索，检索管线决定去哪里搜索和怎么排序， workflow 决定搜完以后怎么与用户互动。

3.6.2 源码节选：候选数量与最终数量分离

代码块 python

```
1 ranker = RRFRanker(k=settings.rrf_k)
2 docs = hybrid_search(
3     query,
4     ranker=ranker,
5     top_k=candidate_k,
6     candidate_k=candidate_k,
7     filter_query=filter_query,
8 )
9 final_docs = cross_encoder_rerank(
10     query,
11     docs,
12     top_k=top_k,
13 )
14 return final_docs
```

两个 top_k 分属不同函数。第一处传的是候选规模，第二处传的是最终精排规模。假设 candidate_k 为 20、top_k 为 5，意思是先给精排最多 20 条候选，再输出最多 5 条，不是先只有 5 条再重排。

源码使用 top_k = top_k or settings.reranker_top_k 这类默认处理。传入 0 时，0 会被视为假值，转而使用默认值；它不会在此表达式之后继续保留为 0。学习 API 参数校验时，不能仅凭下方有 <= 0 检查，就认定显式 0 一定会抛异常。负数为真值，才会继续进入后续检查逻辑。

3.7 Structured Output：把用户问题变成 SearchIntent

3.7.1 为什么还需要“理解查询”

用户说“你好”，不需要查询车辆资料；用户说“徐工 XE135GII 仪表针脚定义”，则包含多个可用于检索的属性。项目先理解输入，再决定走普通聊天还是资料查询。

Structured Output 的意思是结构化输出：让模型按事先约定的字段返回数据，再由程序按结构使用。项目用 Pydantic 的 `SearchIntent` 表示品牌、车型、部件、控制器类型、资料类型和其他关键词。

字段不是模型自行发明的数据库结构，而是应用约定的查询表达。`brand` 等字段可为空，表示无法确认；`keywords` 保存没有被明确字段完整表达的剩余词；`is_search_request` 用于路由。

3.7.2 提取规则中的业务含义

项目提示词要求“只提取用户明确表达的信息”，不能推测品牌和具体型号；型号、数字和字母编码保留原始写法；资料类型放进 `document_type`，仪表或电脑板等部件放进 `component`。

这几个规则尤其重要。假设用户只说“找三一电脑板针脚”，模型不能顺手补成 SY60。把猜测出的车型放入结构化字段后，后面会生成硬过滤条件，正确资料可能被提前排除。

因此结构化输出不仅是把文字变成 JSON。字段怎么定义、哪些信息允许填、哪些值需要留空，都会影响检索结果。一个结构规范但内容猜错的 JSON，依然可能让系统查错。

3.7.3 JSON Mode 的实际使用

下面是 `app/rag/intent_parser.py` 的源码节选。

代码块 python

```
1 structured_model = model.with_structured_output(  
2     SearchIntent,  
3     method="json_mode",  
4 )  
5 intent = structured_model.invoke(  
6     INTENT_PROMPT.format(query=query)  
7 )  
8 return intent if intent is not None else _fallback_intent(query)
```

JSON Mode 让服务按 JSON 形式输出，项目同时在提示词里明确列出 JSON 字段与规则。Pydantic Schema 帮助程序校验和解析数据，但 JSON 形式正确不代表语义一定正确，仍需要业务规则与测试。

这段代码只在解析结果为 `None` 时执行确定性降级。它没有在这里统一捕获所有网络异常、解析异常或字段校验失败。不要把“空响应有回退”写成“所有失败都有安全降级”。两种处理范围不同。

3.7.4 空响应时如何保留用户原意

`_fallback_intent()` 使用一条短问候匹配规则。对“你好！”之类明确短问候，返回 `is_search_request=False`；对其他非空输入，优先视为检索请求，把原问题整体保留为软关键词，不猜测品牌、型号。

这是避免丢失请求的降级策略，不是替代模型理解的完整规则引擎。“你好，帮我找仪表针脚”不属于纯短问候，仍可进入检索。空白输入则在调用模型前直接返回默认 `SearchIntent`；上层 API 对空消息另有校验，不应把空白测试当成真实聊天路线。

完整离线实验。 用空返回的假模型验证降级，不调用 DeepSeek API。

代码块 python

```
1 from app.rag.intent_parser import parse_search_intent
2
3 class EmptyStructured:
4     def invoke(self, text):
5         return None
6
7 class EmptyModel:
8     def with_structured_output(self, schema, *, method):
9         assert method == "json_mode"
10        return EmptyStructured()
11
12 for query in ["你好!", "三一挖掘机针脚定义"]:
13     intent = parse_search_intent(query, model=EmptyModel())
14     print(intent.is_search_request, intent.keywords)
```

对应结果应是 False [] 与 True ['三一挖掘机针脚定义']。本次通过现有测试核验了这两条规则。假模型实验能确认路由逻辑，不能证明真实模型每次都能正确解析复杂查询。

3.8 Query Rewrite: 改写表达，保留具体编码

3.8.1 改写的目标

Query Rewrite 是查询改写。项目把“帮我找东风天龙 EDC17C53 电脑板针脚图”转成更适合检索的关键词表达，尽量剔除无检索意义的口语，同时保留品牌、车型、发动机和 ECU 编码。

源码提示词要求只输出关键词，不解释；核心词以空格分隔；不能补充用户没有提供的具体型号。这种写法较贴近资料目录检索，适合学习“先让查询贴近索引文本，再执行检索”的思路。

不过，当前实现通过提示词要求保留编码，没有另写一个代码级编码校验器。现有测试使用假模型，核验目标字符串和提示要求；它不等于已经证明真实模型永远不把 EDC17C53 改错。面对编码密集的领域，应重点观察改写前后是否遗漏、替换或新增编码。

3.8.2 原始查询为什么还要保存

SearchQueryPlan 同时保留 original_query 与 rewritten_query。前者用于追踪用户确切表达，后者用于执行检索。两者分开保存，才能判断某次漏召回是资料问题、改写问题还是排序问题。

rewrite_query() 在模型返回空内容时回退原始 query。这个回退只解决空字符串情况；若模型返回了非空但错误的编码，当前函数会继续使用该改写文本。调试时应按实际失败类型定位，不能把“有回退”视为改写质量保障。

3.8.3 查询准备与 Web 工作流的对应关系

prepare_search_query() 按意图解析、查询改写、过滤构建的顺序返回 SearchQueryPlan，适合脚本预览标准查询。Web 工作流把同样能力拆到 understand 与 rewrite 两个节点，并在中间根据 is_search_request 分流。

因此，两者使用相关组件，但调用组织方式不同。不要把查询准备脚本的顺序直接写成所有 Web 请求都会完整执行的顺序；普通聊天在 understand 后就进入 chat。

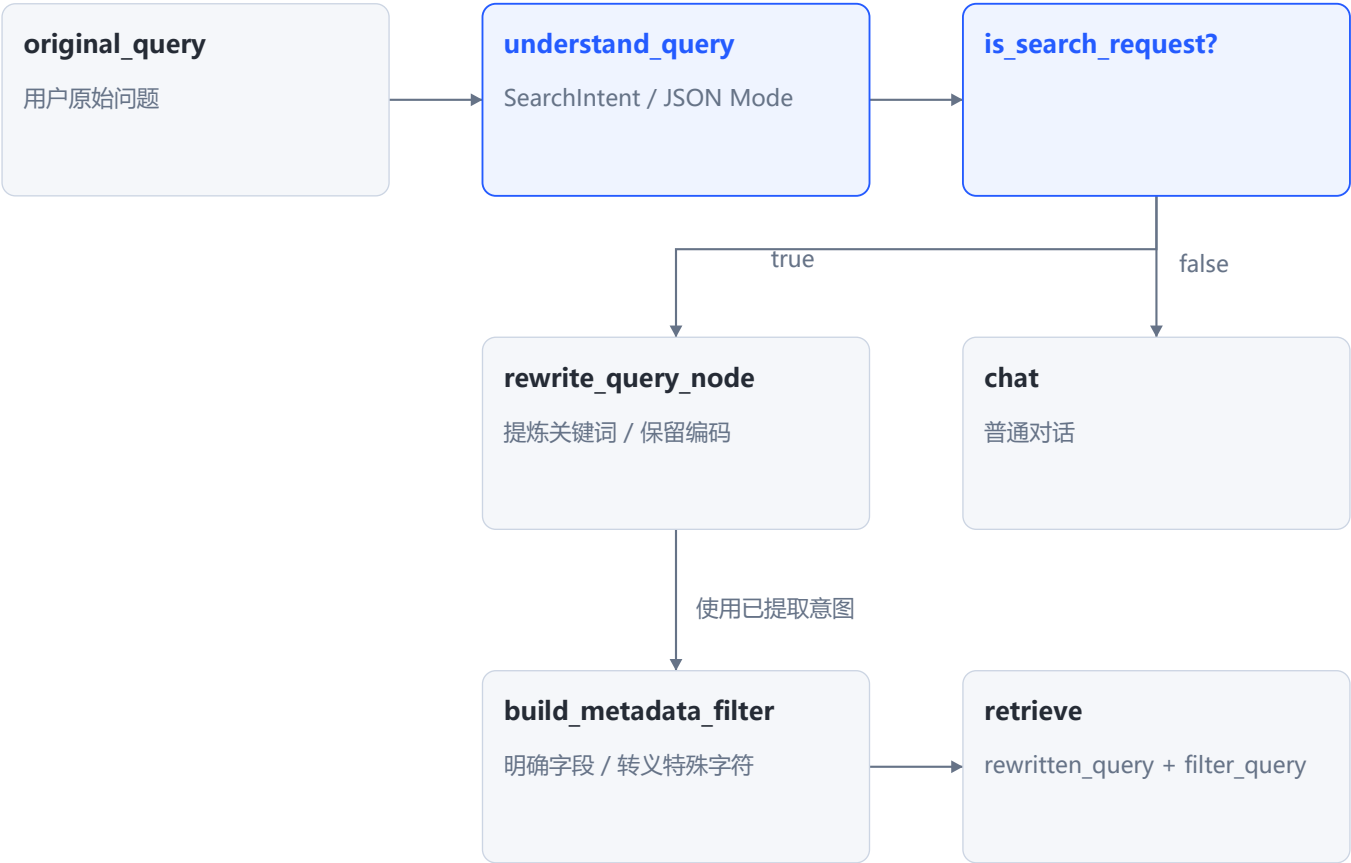


图3-2 Web 工作流的查询准备。普通聊天在理解意图后分流；检索路径保留原始问题并生成改写查询与硬过滤条件。

阅读练习。 分别打开 query_pipeline.py 和 graph/nodes/rewrite.py，寻找原始 query 从哪里读出、意图是否再次调用模型、过滤条件写入哪个字段。能回答这些问题，就开始理解“相同组件被不同入口组织”的工程方式了。

3.9 Metadata Filter：把明确条件变成检索范围

3.9.1 为什么 keywords 不直接加入硬过滤

Metadata Filter 是元数据过滤。当前数据没有把每个业务属性都单独拆成结构化数据库字段，所以代码在 title 与 hierarchy_path 上做 LIKE 条件查询。

品牌、车型、部件、ECU 类型和资料类型会参与过滤。keywords 不直接加入硬过滤，目的是避免软线索过度限制召回。比如用户说“接线”，资料可能叫“针脚定义”，如果把“接线”强行加入标题 LIKE，正确资料就可能全部排除。

每一个明确字段生成“层级路径包含该值，或标题包含该值”的条件。多个字段之间用 AND 连接。只有品牌时范围较宽；再加车型和 ECU 编码，候选范围逐步收窄。

这里的“高置信度”是源码注释描述的使用原则，当前 SearchIntent 并没有一个数值 confidence 字段，也没有概率阈值筛选。模型根据提示词只填明确信息，程序随后使用非空字段；两者要区分。

3.9.2 完整离线实验：看过滤表达式怎样生成

代码块 python

```
1 from app.rag.filters import build_metadata_filter
2 from app.schemas.intent import SearchIntent
3
4 intent = SearchIntent(
5     brand="东风",
6     ecu_type="EDC17C53",
7     keywords=["针脚"],
8 )
9 expr = build_metadata_filter(intent)
10 print(expr)
11 print(build_metadata_filter(
12     SearchIntent(keywords=["针脚"]),
13 ))
```

实际生成的第一个表达式含品牌和 ECU 类型两组条件，组内 OR、组间 AND；不会出现“针脚”。第二个结果为 None，表示没有需要下推给数据库的硬条件。

完整表达式较长，拆开阅读如下。这里仅为同一表达式的换行展示。

代码块 text

```
1 (hierarchy_path LIKE "%东风%" OR title LIKE "%东风%")
2 AND
3 (hierarchy_path LIKE "%EDC17C53%"
4  OR title LIKE "%EDC17C53%")
```

LIKE 是包含匹配，不等于品牌字段的精确等值判断。当前方案依赖标题与路径表达，可能出现同义词不一致、局部字符串误匹配等情况。知道这个限制，就能理解为什么需要下一节的过滤回退。

3.9.3 特殊字符转义为什么必要

用户输入中可能包含引号、反斜杠、百分号或下划线。把未经处理的值直接插入表达式，可能改变表达式结构或扩大 LIKE 的匹配范围。

项目 `_escape_like_value()` 顺序转义反斜杠、双引号、百分号和下划线，再把值放入固定条件模板。百分号和下划线在这里应表示用户文字本身，而不是任意匹配符。外层由程序添加的 % 才用于实现包含匹配。

完整离线实验如下，不连接数据库。

代码块 python

```
1 from app.rag.filters import build_metadata_filter
2 from app.schemas.intent import SearchIntent
3
4 expr = build_metadata_filter(
5     SearchIntent(model='A_10%X'),
6 )
7 print(expr)
```

本次实际打印的值部分为 A_10\%\X，说明_、% 与引号已被转义。这个实验验证字符串构造，不验证某个 Milvus 服务版本的全部表达式行为。真实接入仍需用相应服务版本进行查询验证。

安全阅读还要看完整使用链路：转义解决表达式中特殊字符的含义，固定字段范围限制可构造条件的位置，意图提示限制随意猜测。它们分别处理不同问题，不能把一项措施当作所有输入问题的统一答案。

3.10 过滤回退：精排后零条，才做最终判断

3.10.1 一个容易漏掉的分支

用户想找“徐工仪表线路图”，目录里可能只有“徐工 XE135GII 仪表显示器针脚定义”。如果意图解析将线路图作为明确资料类型，LIKE 硬过滤可能挡住后者。

Web retrieve 节点先带过滤做混合召回，再做精排。只有存在过滤条件，并且精排后的结果为空，才放弃本次过滤，重新做无过滤混合检索与精排。

关键是“精排后的结果为空”。过滤召回有 1 条弱相关记录，并不表示流程已经成功；这条记录可能被精排阈值淘汰。此时如果只检查召回阶段有没有内容，就不会触发回退，用户仍看不到可能相关的资料。

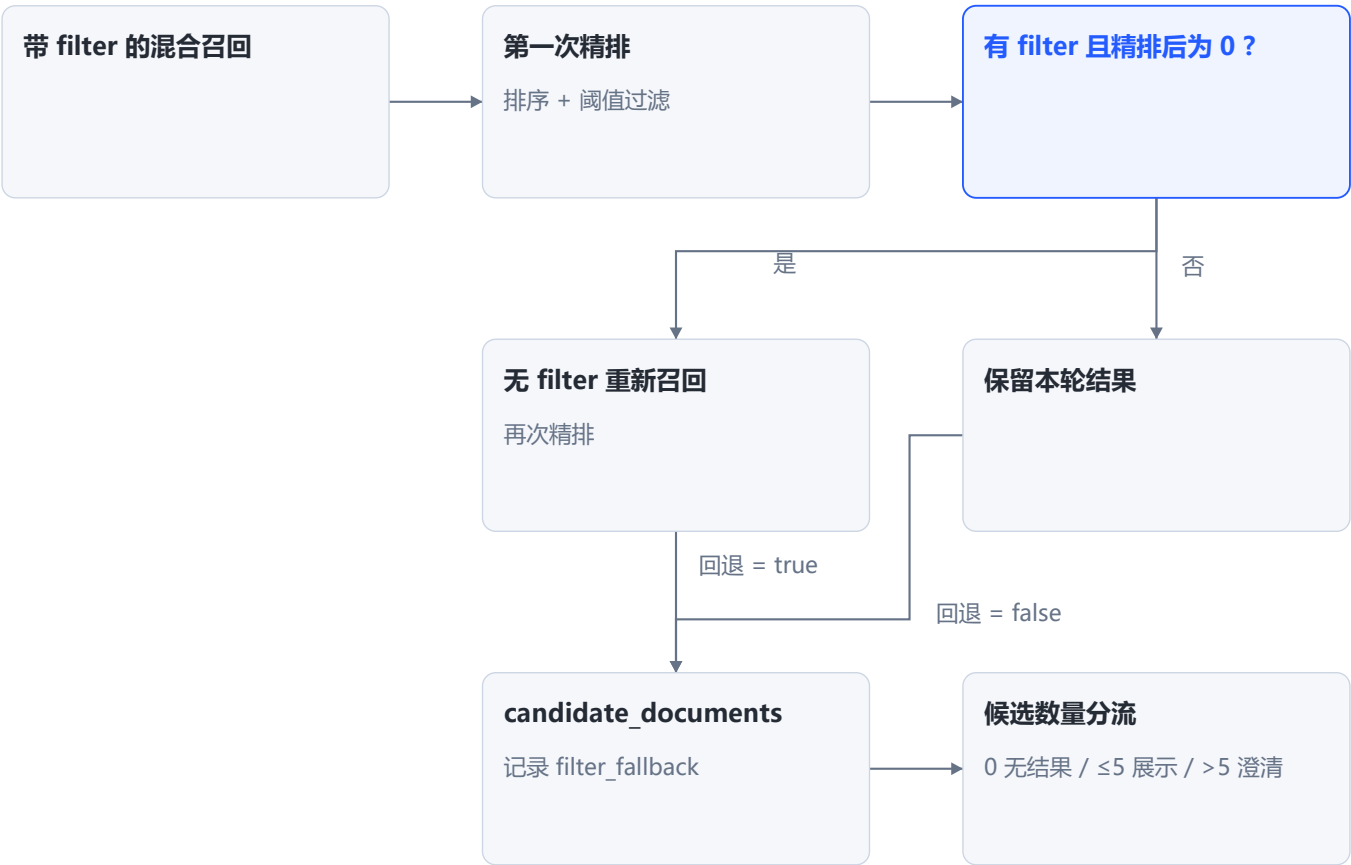


图3-3 精排后零命中的过滤回退。条件判断位于第一次精排之后。带过滤召回非空、精排全淘汰时，同样会执行无过滤重查。

3.10.2 源码节选：回退条件与再次检索

代码块 python

```
1 filter_fallback = bool(filter_query and not docs)
2 if filter_fallback:
3     fallback_docs = hybrid_search(
4         query,
5         ranker=ranker,
6         top_k=settings.graph_candidate_k,
7         candidate_k=settings.graph_candidate_k,
8         filter_query=None,
9     )
10 docs = cross_encoder_rerank(
11     query,
12     fallback_docs,
13     top_k=settings.graph_candidate_k,
14     min_score=settings.reranker_min_score,
15 )
```

这里的 docs 已经是第一次精排后的结果。再次请求去掉 filter，查询文本、RRF 常数、候选规模和精排阈值保持同一套设置。回退只改变过强的硬约束，不是在没有证据时让模型编一条资料。

结果状态中记录 filter_fallback=True，并把 filter_query 清空；下游仍按候选数量决定无结果、直接展示或分类澄清。如果无过滤的第二轮仍为零，就进入无结果分支。

3.10.3 这个保障只在哪个入口生效

情况	Web retrieve 节点的处理	需要观察什么
无过滤且精排为空	直接进入无结果分支	查询、索引与分数阈值
有过滤且召回为空	再做无过滤召回和精排	标题表达与过滤条件
有过滤、召回非空、精排全淘汰	同样执行无过滤回退	精排阈值及回退后候选
有过滤且精排有结果	保留本轮结果	候选是否符合原始需求
无过滤回退后仍为空	最终无结果	是否确实缺少对应资料

回退逻辑位于 app/graph/nodes/retrieve.py。通用 retrieve_docs() 本身没有复制这段零命中回退。学习时如果把两个入口混为一谈，就会错误期待命令行检索脚本自动执行同样分支。

回退也会扩大候选范围，可能产生更弱的匹配。因此后续仍需要精排、分类澄清或明确说明无结果。不能宣称“加了回退就再也搜不到空结果”。它主要降低因标题表达差异导致的假阴性。

3.11 实验、调试与输出解释

3.11.1 先验证纯函数，再运行模型服务

第一层先验证 RRF 计算和过滤表达式；第二层用假模型验证意图降级和精排排序；第三层才连接真实 Milvus、Embedding、Reranker 与聊天服务。这样的顺序可以把数据处理错误、模型效果和服务故障分别定位。

本次执行了上述 RRF 与过滤构造实验，并运行了仓库现有的融合、过滤、查询改写、意图解析、精排和检索回退测试，结果为 13 项通过。它们验证的是算法与分支行为，没有发起新的模型 API 请求，也没有下载模型或重建资料索引。

3.11.2 读者自行运行的入口

下面命令是读者运行入口，不是本次已执行的在线检索结果。执行前需要项目虚拟环境、相应索引和本地模型；意图与改写预览还需要配置聊天模型服务。

代码块 powershell

```
1 python -m scripts.prepare_query "东风天龙EDC17C53针脚定义"
2 python -m scripts.search_dense "EDC17C53" --top-k 5
3 python -m scripts.search_sparse "EDC17C53" --top-k 5
4 python -m scripts.search_hybrid "东风 ECU 针脚" --top-k 10
5 python -m scripts.search_rerank "东风 ECU 针脚" --top-k 5
```

运行 prepare_query 时，应逐项核对 original、rewritten、intent 与 filter。若编码在 rewritten 中改变，先调查改写；若 filter 加入用户未说过的车型，先调查意图；若两者都正确，再检查索引和召回。

运行 search_rerank 时，输出同时包含 rerank、retrieval、id、title 和 path。两个分数分别对应精排和融合，不要跨阶段比较大小。检查命中的资料 ID 是否稳定，标题与路径是否能说明品牌、车型、部件，排名靠前是否只是名称看似相关。

3.11.3 一个实用的定位顺序

当用户说“查不到”，先保留原始问题，再查看改写文本，接着查看过滤表达式；确定查询输入后，对比 Dense、BM25 的候选与融合结果；最后检查精排之前和之后的数量。

当用户说“查到了但不”，先检查型号和资料类型，不要只看语义相似；随后观察目标资料有没有进入候选，确认是召回缺失还是排名靠后；如果 Web 候选很多，再检查澄清流程是否正确缩小范围。

记录数量至少应包括每路召回量、融合候选量、精排保留量和展示量。只记录最终 5 条，往往无法区分“本来就只有 5 条”和“30 条中截取了 5 条”。真实效果评估需要一批有标签的查询，留到评估章节继续学习。

3.12 常见问题与易错点

3.12.1 有 Dense，为什么还需要 BM25

因为语义相似与明确词项匹配提供不同证据。编码、品牌和术语对资料检索很重要，单一路线可能漏掉某类查询。是否值得使用混合检索，应通过真实查询集比较，而不是认定混合一定优于所有单路方案。

3.12.2 RRF 得分只有 0.03，是不是相关性很低

不能这样判断。RRF 分数由排名和平滑常数产生。默认 k 为 60，两路首名贡献相加也只是约 0.0328。这个量用于融合排序，不能套用聊天模型概率或余弦分数的直觉。

3.12.3 Reranker 可以找回被过滤的资料吗

它只处理传入的候选列表。如果过滤阶段删除了目标记录，精排模型看不到它。Web 的无过滤回退会重新召回，给这类记录再次参与排名的机会，但也只在规定的零结果条件下发生。

3.12.4 Structured Output 能防止模型乱猜吗

它主要约束输出格式与字段结构。提示规则能指导模型不猜测，实际效果仍需验证。填入一个结构合法但错误的品牌，程序照样可能生成错误过滤条件。

3.12.5 改写后的关键词越多越好吗

不一定。添加用户没提供的型号可能让 Dense 偏向错误候选，或让 BM25 被错误词影响。这个项目的策略是提炼原问题而不是补全猜测，尤其强调保留字母数字编码。

3.12.6 min_score=0 表示资料无关的概率为零吗

不是。它只表示过滤比较使用 0 这个数。分数可能来自 logits、归一化概率或具体模型的其他约定。先核对模型与适配层，再通过数据选择阈值。假模型分数只能验证排序和过滤代码。

3.12.7 候选设为 30，为什么最终还有澄清步骤

30 是为保留候选设置的上限。精排后仍超过默认 5 条，工作流认为直接全部展示不够聚焦，进入分类澄清。第5章将学习如何从候选路径和资料类型生成选项。

3.12.8 学这章时最容易写错什么

最常见的错误是把配置默认值当作所有入口的实际参数，把 RRF 与精排的 score 当成同一尺度，把 JSON 输出当作语义正确证明，把 query rewrite 的提示约束当作代码级编码保证，以及把 Web 过滤回退写成通用检索函数的共同能力。

纠正方式很简单：对每项结论都找到对应的调用点、传参和返回结构。能指出某行代码为何导致某种行为，比只背“Dense + Sparse + Rerank” 更有价值。

3.13 本章练习与源码阅读索引

3.13.1 三个学习任务

任务一：手算融合。 在第 3 节实验中，删掉 sparse 的资料 1，先手算新的排名，再执行代码核对。解释为什么两路共同命中的资料通常有优势。记录公式和输出，避免只保存一张运行截图。

任务二：检查查询计划。 选择五条问题：纯问候、只有部件、明确品牌车型、包含控制器编码、包含百分号或下划线。先写出你认为应生成的意图与过滤条件，再核对函数行为。真实模型没有运行时，把预期写为待验证，不要伪造模型输出。

任务三：解释回退。 阅读两个检索回退测试，说明“第一次无候选”与“第一次有候选但精排全淘汰”为什么都会重查。再解释第二次为什么仍要精排。如果能独立画出这两条路径，你就能讨论该逻辑的覆盖范围。

3.13.2 建议阅读顺序

按下面顺序阅读，先形成检索主线，再学习查询准备，最后理解 Web 的额外保障。路径均相对项目根目录。

源码位置	阅读重点
app/schemas/search.py	四种结果类型与两类最终分数
app/rag/fusion.py	按 doc_id 聚合，按排名贡献计算
app/rag/hybrid_search.py	两份请求、RRF 对象、响应映射
app/rag/reranker.py	模型复用、pairs、排序、严格阈值
app/rag/pipeline.py	通用检索串联与数量参数
app/schemas/intent.py	查询意图字段与空值语义
app/rag/intent_parser.py	JSON Mode 提示与 None 降级
app/rag/query_rewriter.py / filters.py	保编码要求与条件转义
app/graph/nodes/retrieve.py	精排后零命中与无过滤回退

阅读本机依赖实现时，再看 .venv/Lib/site-packages/sentence_transformers/cross_encoder/model.py 与 cross_encoder/modules/logit_score.py。这是了解当前安装库的参考路径，不是要求你修改第三方库。

3.13.3 如何形成可用于复盘的项目表达

学完后，尝试用三句话说明项目：第一句说明资料目录查询遇到的语义表达与编码匹配问题；第二句说明两路召回、RRF 融合和精排的具体分工；第三句说明意图解析、查询改写与 Web 零命中过滤回退怎样处理输入和边界。

不要在尚未评估时填写召回率提升百分比，也不要把目录检索包装成已读取电路图全部内容的故障诊断系统。真实的技术贡献应来自你能解释、运行和验证的部分。

下一章将把检索能力封装成 Agent 工具；第5章再学习 workflow 状态、候选分类、interrupt 与 resume。它们接手本章输出的 candidate_documents，解决“候选很多时如何逐步帮助用户选择”。

3.13.4 官方资料与验证边界

补充阅读：[Milvus 全文检索](#)、[Milvus 多向量混合检索](#)、[Milvus RRF 排名器](#)、[Qwen3 Reranker 官方模型卡](#)、[DeepSeek JSON Mode](#)。

这些资料解释组件原理与接口；项目行为仍以当前源码和安装依赖为准。本章未执行模型下载、索引重建、真实聊天服务调用或真实 Qwen 推理。离线实验和现有测试结果用来确认算法与分支，真实检索质量需在后续评估章节用带标签的查询集衡量。